

GUÍA COMPLETA DE QUALITY ASSURANCE PARA MÓDULOS DE ODOO

Testing Funcional y Técnico

- ✓ Guía para Consultores
- ✓ Guía para Desarrolladores
- ✓ Casos Prácticos y Ejemplos
- ✓ Checklists Completos

Sunname Technologies

Versión 1.0 - 2026

TABLA DE CONTENIDOS

INTRODUCCIÓN

Este manual ha sido diseñado específicamente para Sunname Technologies con el objetivo de garantizar la máxima calidad en los módulos de Odoo desarrollados por terceros. Como empresa joven en crecimiento, es fundamental contar con procesos de Quality Assurance (QA) robustos que nos permitan validar exhaustivamente cada desarrollo antes de implementarlo en producción.

El Contexto de Sunname Technologies

Actualmente, Sunname Technologies ofrece servicios de desarrollo de módulos personalizados para Odoo, pero al no contar aún con un equipo de desarrollo interno de gran tamaño, dependemos de desarrolladores externos (freelancers, agencias, partners) para ejecutar los proyectos.

Esta situación presenta desafíos específicos:

- Los desarrolladores externos pueden tener diferentes niveles de experiencia con Odoo
- No siempre siguen las mejores prácticas o estándares de la plataforma
- Los módulos pueden funcionar en apariencia, pero contener errores ocultos que se manifiestan en producción
- La calidad del código puede variar significativamente entre desarrolladores
- Los problemas de rendimiento, seguridad o compatibilidad pueden no ser evidentes en las pruebas iniciales

Objetivo de Este Manual

Este manual tiene tres objetivos principales:

1. Estandarizar el proceso de validación: Establecer protocolos claros y reproducibles que cualquier miembro del equipo pueda seguir.
2. Detectar errores antes de producción: Identificar problemas funcionales, técnicos, de rendimiento o seguridad en fase de desarrollo o staging, antes de que afecten a los usuarios finales.
3. Educar al equipo: Proporcionar conocimiento detallado sobre cómo funciona Odoo internamente, qué buscar en las revisiones y cómo interpretar los resultados de las pruebas.

Estructura del Manual

El manual está dividido en cuatro partes principales:

- Parte 1 - Guía para Consultores: Enfocada en el testing funcional desde la perspectiva del usuario. No requiere conocimientos de programación.

- Parte 2 - Guía para Desarrolladores: Cubre el análisis técnico del código, arquitectura, rendimiento y seguridad.
- Parte 3 - Procesos y Herramientas: Metodologías de trabajo, ambientes de prueba, automatización y mejores prácticas de equipo.
- Parte 4 - Casos de Estudio: Ejemplos reales de errores comunes, cómo detectarlos y cómo prevenirlos.

 Principio Fundamental

El objetivo no es solo encontrar errores, sino prevenirlos. Un buen proceso de QA debe convertirse en parte de la cultura del equipo, no solo en una lista de verificación al final del desarrollo.

PARTE 1: GUÍA PARA CONSULTORES

Testing Funcional sin Conocimientos Técnicos

Como consultor o Project Manager, tu rol es fundamental en el proceso de Quality Assurance. Aunque no tengas conocimientos de programación, eres el guardián de la experiencia del usuario y el representante del cliente. Debes pensar como el usuario más despistado y, simultáneamente, como el auditor más estricto.

1. Experiencia de Usuario (UI/UX)

La interfaz de usuario es el punto de contacto directo entre el sistema y las personas. Un módulo puede tener código perfecto, pero si la interfaz es confusa o poco intuitiva, el proyecto fracasará.

1.1 Vistas de Búsqueda y Filtros



Concepto Clave

Si un campo existe en el formulario, debe ser buscable y filtrable. El usuario no debería tener que abrir 50 registros para encontrar el que necesita.

Pruebas específicas:

- Navega al modelo/vista donde se agregó el nuevo campo (por ejemplo, si se agregó 'Tipo de Cliente' en Contactos)
- Haz clic en el icono de búsqueda (lupa) en la parte superior derecha
- Verifica que el nuevo campo aparezca en 'Filtros' y en 'Agrupar por'
- Prueba filtrar por diferentes valores del campo
- Prueba agrupar por el campo y verifica que los grupos se muestren correctamente

Ejemplo práctico:

Si agregaste un campo 'Sector Industrial' en los clientes con valores como 'Retail', 'Manufactura', 'Servicios', deberías poder:

- Filtrar solo clientes de 'Retail'
- Agrupar todos los clientes por sector y ver cuántos hay en cada uno
- Combinar filtros (Clientes de 'Retail' que sean de 'México')

Errores comunes a detectar:

- El campo no aparece en la búsqueda avanzada
- Los filtros no funcionan correctamente (muestran resultados incorrectos)
- Agrupar por fechas no respeta el formato (muestra fechas exactas en lugar de mes/año)
- Los campos calculados no se pueden filtrar (falta configuración store=True)

1.2 Vistas de Lista (Tree View)

La vista de lista es la primera impresión que un usuario tiene de los datos. Debe mostrar información relevante sin ser abrumadora.

Aspectos a validar:

- Información esencial visible: ¿Los campos más importantes se ven sin tener que abrir cada registro?
- Número de columnas: ¿La vista tiene un número razonable de columnas (idealmente 5-8)? Si tiene más de 10, el usuario tendrá que hacer scroll horizontal constantemente.
- Orden de columnas: ¿Las columnas están ordenadas de forma lógica (las más importantes primero)?
- Totales y agregaciones: Si la vista muestra importes, cantidades o números, ¿hay una suma total en la parte inferior?
- Colores y badges: ¿Se usan colores para resaltar estados importantes (ej: rojo para vencido, verde para pagado)?

Ejemplo de prueba: 1. Ve a Ventas > Órdenes de Venta 2. Verifica que veas: Número, Cliente, Fecha, Total, Estado 3. Verifica que NO tengas que hacer scroll horizontal 4. Verifica que la columna 'Total' muestre una suma al final 5. Verifica que los estados tengan colores distintivos

1.3 Vistas de Formulario

El formulario es donde ocurre la creación y edición de datos. Debe ser intuitivo y eficiente.

Elementos a verificar:

- Organización lógica: ¿Los campos están agrupados de forma coherente? (datos generales, direcciones, datos fiscales, etc.)
- Labels claros: ¿Los nombres de los campos son comprensibles para el usuario final? Evitar términos técnicos innecesarios.
- Campos requeridos: ¿Los campos obligatorios están marcados claramente? ¿El sistema previene guardar sin completarlos?
- Ayudas contextuales: ¿Hay tooltips (ayudas emergentes) en campos complejos?
- Placeholder adecuados: ¿Los campos de texto tienen ejemplos o indicaciones?

1.4 Valores por Defecto

Principio de Eficiencia

Si un usuario tiene que seleccionar 'México' como país en cada registro que crea, estás desperdiciando tiempo de tu equipo. Los valores obvios deben estar precargados.

Casos comunes donde deben existir defaults:

- Moneda de la compañía (MXN para empresas mexicanas)
- País predeterminado

- Fecha actual en campos de fecha
- Usuario actual en campos de responsable
- Almacén predeterminado
- Lista de precios activa

Prueba práctica: 1. Crea un nuevo registro (Cliente, Venta, Factura, etc.) 2. Observa qué campos ya están llenos 3. Pregúntate: ¿Los valores precargados tienen sentido? 4. Intenta crear 5 registros seguidos 5. Si repites el mismo valor más de 3 veces, debería ser un default

2. Flujos de Negocio y Lógica

Aquí es donde la mayoría de los errores críticos se esconden. No basta con que el 'camino feliz' funcione; debes probar todos los escenarios, especialmente los que el usuario no debería hacer pero inevitablemente hará.

2.1 El 'Happy Path' (Camino Feliz)

Este es el flujo ideal donde todo sale bien. Es lo primero que debes probar, pero NO lo único.

Ejemplo - Flujo de Venta:

4. Crear cotización
5. Agregar productos
6. Confirmar cotización (convertir a orden)
7. Validar entrega
8. Crear factura
9. Registrar pago
10. Verificar que el estado cambie a 'Pagado'

Si todos estos pasos funcionan correctamente, el Happy Path está OK. Pero aquí empieza lo interesante...

2.2 El 'Unhappy Path' (Caminos de Error)



Mentalidad de Tester

Tu trabajo es romper el sistema. Si no puedes romperlo tú en desarrollo, un usuario lo romperá en producción. Sé creativo pensando en escenarios problemáticos.

Escenarios críticos a probar:

- Cancelación a mitad de flujo
- Reversión de estados (volver de 'Confirmado' a 'Borrador')
- Eliminación de registros relacionados
- Edición después de confirmación
- Duplicación de registros
- Cambio de valores críticos (precio, cantidad) después de procesar

Ejemplo real - Cancelación de factura:

Prueba: 1. Crea una factura y valídala 2. Cancélala usando el botón 'Cancelar' 3. Intenta volver a ponerla en borrador 4. ¿Qué pasa con los movimientos contables? 5. ¿Se liberó el número de folio? 6. ¿Los impuestos se revirtieron? 7. ¿El XML CFDI se marcó como cancelado en el SAT? Error común: El sistema permite regresar a borrador pero deja "basura" contable o duplica folios.

2.3 Validaciones y Restricciones

El sistema debe prevenir que el usuario haga cosas ilógicas o peligrosas.

Validaciones comunes a verificar:

Tipo de Validación	Qué Probar	Error Común
Stock	Intentar vender más de lo disponible	Permite stock negativo
Fechas	Fecha entrega antes de fecha pedido	No valida lógica temporal
Números	Ingresar valores negativos, cero, decimales	Acepta -100 en cantidad
Duplicados	Crear dos registros con mismo código/RFC	No valida unicidad
Permisos	Usuario sin permisos puede editar	Falta record rules

2.4 Integridad Referencial

¿Qué pasa cuando eliminas un registro que está relacionado con otros? El sistema debe manejarlo correctamente.

Escenarios a probar:

- Eliminar un cliente que tiene facturas pendientes: ¿El sistema lo previene o permite crear huérfanos?
- Eliminar un producto que está en órdenes de venta: ¿Se bloquea o se corrompe la data?
- Cambiar la categoría de un producto que está en análisis de costos: ¿Se actualiza todo automáticamente?



Regla de Oro

NUNCA debe ser posible eliminar un registro 'padre' si tiene registros 'hijos' activos, a menos que se eliminen en cascada de forma controlada.

3. Localización Mexicana (Facturación Electrónica - SAT)

Si el módulo toca de alguna forma el proceso de ventas, compras o inventarios, DEBE validarse su impacto en la facturación electrónica. Un error aquí puede significar rechazo del SAT y problemas legales para el cliente.

3.1 Validación de XML (CFDI 4.0)

Proceso de validación:

11. Crea una venta/compra que utilice el nuevo módulo
12. Genera la factura electrónica
13. Timbra el CFDI (en ambiente de pruebas del PAC)
14. Descarga el XML
15. Valida el XML contra el esquema del SAT

Aspectos críticos a verificar:

- Estructura correcta: ¿El XML tiene todos los nodos requeridos (Emisor, Receptor, Conceptos, Impuestos)?
- Cálculos exactos: ¿Los subtotales, IVA, retenciones y total coinciden al centavo?
- Redondeos: ¿No hay diferencias de 1 o 2 centavos que causen rechazo?
- Catálogos SAT: ¿Las claves de producto, unidad, uso CFDI son válidas?
- Complementos: Si usa complementos (nómina, pagos, comercio exterior), ¿están bien formados?

Error común - Porcentaje de impuestos: **✗ MAL:** <Impuesto TasaOCuota="16" ...> **✓ BIEN:** <Impuesto TasaOCuota="0.160000" ...> El SAT requiere formato decimal, no porcentaje entero.

3.2 Datos Fiscales Obligatorios

Verifica que el sistema no permita timbrar sin:

- RFC válido del cliente (12-13 caracteres)
- Régimen fiscal del cliente
- Código postal del cliente
- Uso de CFDI seleccionado
- Forma de pago y método de pago
- Productos con clave SAT asignada



Validación de RFC

El RFC debe validarse con checksum. No basta con contar caracteres. Ejemplo: 'XAXX010101000' es válido como RFC genérico público, pero 'ABCD123456ABC' aunque tenga 13 caracteres, es inválido.

4. Reportes y Documentos

Los reportes son la ventana de información del sistema. Deben ser precisos, legibles y exportables.

4.1 Reportes PDF

Aspectos a validar:

- Formato profesional: Logos, colores corporativos, alineación correcta
- Datos completos: Toda la información relevante debe estar presente
- Paginación: ¿El reporte maneja múltiples páginas? ¿Los headers/footers se repiten?
- Saltos de página: ¿No corta tablas o secciones en lugares ilógicos?
- Totales y subtotales: ¿Las sumas son correctas?
- Filtros aplicados: ¿El reporte respeta los filtros de fecha/cliente/producto?

4.2 Reportes Excel

Verifica:

- Formato correcto de celdas (números como números, fechas como fechas)
- Encabezados en la primera fila
- Sin filas vacías innecesarias
- Fórmulas funcionales (si aplica)
- Exporta TODOS los registros (no solo los de la página actual)

Prueba práctica: 1. Genera un reporte con 100+ registros 2. Exporta a Excel 3. Abre el archivo 4. Cuenta las filas (debe haber 100+ más header) 5. Suma una columna numérica manualmente 6. Compara con el total del sistema

5. Seguridad y Permisos

La seguridad funcional se refiere a qué puede ver y hacer cada tipo de usuario. No confundir con seguridad técnica (SQL injection, etc.) que se cubre en la Parte 2.

5.1 Pruebas con Diferentes Roles

Debes probar el módulo con al menos estos perfiles:

16. Administrador (acceso total)
17. Usuario regular (permisos estándar)
18. Usuario de solo lectura
19. Usuario externo (portal)

Para cada perfil, verifica:

- ¿Ve solo los menús que debería ver?
- ¿Puede crear/editar solo lo que se le permite?

- ¿Los botones de acción peligrosos están ocultos? (Eliminar, Cancelar, etc.)
- ¿Puede ver datos de otras compañías/equipos si es multi-compañía?

5.2 Trazabilidad (Chatter)

El Chatter es el registro de auditoría de Odoo. Debe registrar todos los cambios importantes.

Prueba: 1. Abre un registro (Cliente, Venta, etc.) 2. Cambia un valor importante (Límite de Crédito, Precio, Estado) 3. Guarda 4. Revisa el Chatter en la parte inferior 5. Debe aparecer: "[Usuario] cambió [Campo] de [Valor Anterior] a [Valor Nuevo]"

Seguridad Crítica

Si cambios en precios, límites de crédito, descuentos o datos fiscales NO quedan registrados en el Chatter, hay un riesgo de fraude interno. Rechaza el módulo hasta que se corrija.

6. Compatibilidad Multi-Dispositivo

Aunque Odoo está optimizado para desktop, muchos usuarios acceden desde tablets o celulares. Las vistas deben ser al menos funcionales en móvil.

6.1 Pruebas en Móvil

Aspectos a verificar:

- Los botones son suficientemente grandes para tocar (no requieren precisión de cursor)
- Las tablas no se desbordan horizontalmente
- Los formularios son navegables sin hacer zoom
- Los dropdowns y selects funcionan correctamente
- El chatter es accesible (no queda cortado)

Cómo probar:

20. Usa el modo responsivo del navegador (F12 > Toggle Device Toolbar)
21. Prueba en resoluciones: 375x667 (iPhone), 360x740 (Android), 768x1024 (iPad)
22. Navega por el módulo completo en cada resolución
23. Intenta completar el flujo principal (crear, editar, guardar)

7. Checklist Final para Consultores

Usa esta tabla como guía rápida antes de aprobar cualquier módulo:

Aspecto a Validar	Estado
1. Búsqueda y filtros funcionan correctamente	☐ OK / ☐ Falla
2. Vista de lista es clara y no requiere scroll horizontal	☐ OK / ☐ Falla
3. Formularios tienen valores por defecto lógicos	☐ OK / ☐ Falla
4. Happy path funciona de inicio a fin	☐ OK / ☐ Falla
5. Cancelaciones y reversiones no dejan datos corruptos	☐ OK / ☐ Falla
6. Validaciones previenen datos ilógicos (fechas, negativos, duplicados)	☐ OK / ☐ Falla
7. XML CFDI se genera correctamente (si aplica)	☐ OK / ☐ Falla / ☐ N/A
8. Reportes PDF/Excel son precisos y exportables	☐ OK / ☐ Falla
9. Permisos por rol funcionan correctamente	☐ OK / ☐ Falla
10. Chatter registra cambios en campos críticos	☐ OK / ☐ Falla
11. Funciona correctamente en móvil/tablet	☐ OK / ☐ Falla
12. Traducciones en español están completas	☐ OK / ☐ Falla
⚠ Regla de Aprobación Si más de 2 ítems de este checklist fallan, RECHAZA el módulo y solicita correcciones. No comprometas la calidad por cumplir tiempos. Un módulo defectuoso en producción cuesta mucho más que unos días extra de desarrollo.	

```
# Ejemplo de manifest bien estructurado {
  'name': 'Gestión Avanzada de
Contratos',
  'version': '16.0.1.0.0',
  'category': 'Sales',
  'summary': 'Gestión integral de contratos con renovación automática',
  'description': ''
    Módulo completo para gestión de contratos que
incluye:
    * Renovación automática
    * Alertas de vencimiento
* Histórico de versiones
* Integración con facturación
'',
  'author': 'Sunname Technologies',
  'website': 'https://www.sunname.com',
  'license': 'LGPL-3',
  'depends': [
    'base',
    'sale_management',
    'account',
    ],
  'data':
[
  'security/ir.model.access.csv',
  'security/record_rules.xml',
  'data/secuencias.xml',
  'views/reporte_views.xml',
  'views/menu.xml',
  'report/reporte_contrato.xml',
  ],
  'demo': [
    'demo/demo_contratos.xml',
    ],
  'installable': True,
  'application':
False,
  'auto_install': False,
}
```



Validación Crítica - Dependencias

Si el módulo importa algo de 'sale_management' pero no lo declara en depends, funcionará solo si sale está instalado. Esto causará errores impredecibles. SIEMPRE verifica que todas las dependencias estén declaradas.

Puntos a validar en el manifest:

- Version sigue el formato: {odoo_version}.{major}.{minor}.{patch}
- Depends lista TODOS los módulos necesarios (verifica imports en Python)
- Data tiene el orden correcto (security → data → views → reports)
- License es apropiada (LGPL-3 para código compatible, OPL-1 para propietario)
- Application es False (a menos que sea un módulo principal como CRM)

2. Calidad del Código Python

2.1 Estilo y Convenciones (PEP 8)

Odoo sigue las convenciones de Python (PEP 8) con algunas particularidades.

Convenciones obligatorias:

- Nombres de clases en PascalCase: SaleOrder, AccountMove
- Nombres de métodos y variables en snake_case: compute_total, partner_id
- Constantes en UPPER_SNAKE_CASE: DEFAULT_DISCOUNT, MAX_QUANTITY
- Líneas máximo 79 caracteres (PEP 8 estricto) o 120 (flexible)
- Imports ordenados: stdlib → third-party → odoo → relative

```
# ✓ CORRECTO import json import logging from datetime import datetime,
timedelta from odoo import api, fields, models, _ from odoo.exceptions
import UserError, ValidationError _logger = logging.getLogger(__name__)
class SaleOrder(models.Model): _inherit = 'sale.order'
custom_discount = fields.Float(string='Descuento Personalizado',
digits=(16, 2), default=0.0, )
```

2.2 Herencia Correcta

Odoo utiliza un sistema de herencia múltiple. Es fundamental entender cuándo usar cada tipo.

Tipo	Cuándo Usar	Ejemplo
<code>_inherit</code>	Extender modelo existente	Agregar campo a sale.order
<code>_name + _inherit</code>	Nuevo modelo basado en existente	Crear modelo reutilizando lógica mail.thread
<code>_inherits</code>	Delegar campos a otro modelo	res.users hereda de res.partner

2.3 Uso Correcto de super()



Error Crítico Común

No llamar a super() en métodos CRUD (create, write, unlink) es uno de los errores más graves. Rompe la contabilidad, los logs, las secuencias y puede causar corrupción de datos.

```
# ✗ MAL - Sobrescribe sin llamar original def create(self, vals):
vals['custom_field'] = 'valor' return self # FALTA super() # ✓ CORRECTO
- Llama al método padre @api.model def create(self, vals):
vals['custom_field'] = 'valor' return super(MiModelo, self).create(vals)
# ✓ MEJOR - Sintaxis moderna Python 3 @api.model def create(self, vals):
vals['custom_field'] = 'valor' return super().create(vals)
```

Otros métodos que SIEMPRE requieren super():

- `create()`, `write()`, `unlink()`
- `_compute_*`() si heredas un campo `computed`
- `action_*`() (`action_confirm`, `action_cancel`, etc.)
- `default_get()`

3. Rendimiento y Optimización

Un código que funciona pero es lento es casi tan malo como un código que no funciona. El rendimiento debe ser considerado desde el diseño.

3.1 Problema N+1 (Queries en Bucles)

Este es EL problema de rendimiento más común en módulos Odoo. Ocurre cuando haces consultas a la base de datos dentro de un bucle.

```
# ❌ MALO - N+1 Problem (1 query + N queries) for order in sale_orders: # 1
query para obtener órdenes      partner =
self.env['res.partner'].browse(order.partner_id.id) # N queries
print(partner.name) # Si hay 1000 órdenes = 1 + 1000 = 1001 queries 🤯 # ✓
BIEN - Prefetching automático de Odoo for order in sale_orders:
print(order.partner_id.name) # Odoo precarga automáticamente # Solo 2
queries totales (órdenes + partners) ✓ # ▢ MEJOR - Búsqueda en lote explícita
orders = self.env['sale.order'].search([('state', '=', 'sale')]) partner_ids
= orders.mapped('partner_id') # Ahora partner_ids está precargado en memoria
for order in orders:      print(order.partner_id.name) # No genera queries
adicionales
```



Herramienta de Detección

Activa el modo debug en Odoo y usa el profiler o revisa el log de SQL. Si ves el mismo tipo de query repetido 100+ veces con IDs diferentes, tienes un problema N+1.

3.2 Campos Calculados (Computed Fields)

Los campos computed son poderosos pero pueden destruir el rendimiento si se usan mal.

Reglas de oro:

- Si el campo se consulta frecuentemente, usa store=True (lo guarda en BD)
- Si usas store=True, DEBES declarar @api.depends() correctamente
- Nunca hagas búsquedas (search) dentro de un compute sin store=True
- Evita depends de campos que cambian constantemente (now(), user_id, etc.)

```
# ❌ MAL - Compute sin depends, causará cálculos incorrectos @api.depends()
# Vacío! def _compute_total(self):      for record in self:
record.total = record.price * record.quantity # ✓ CORRECTO - Depends
completo @api.depends('price', 'quantity') def _compute_total(self):      for
record in self:      record.total = record.price * record.quantity # ✓
MEJOR - Con store para consultas rápidas total =
fields.Float(      compute='_compute_total',      store=True, # Se guarda en
BD      index=True, # Se puede buscar eficientemente ) @api.depends('price',
'quantity', 'discount') def _compute_total(self):      for record in self:
subtotal = record.price * record.quantity      record.total = subtotal *
(1 - record.discount / 100)
```

3.3 Índices SQL

Si un campo se usa frecuentemente en búsquedas o filtros, debe tener index=True.

```
# Campos que DEBERÍAN tener index=True: reference =  
fields.Char(string='Referencia', index=True) # Búsquedas frecuentes state =  
fields.Selection(..., index=True) # Filtros por estado partner_id =  
fields.Many2One('res.partner', index=True) # Relaciones date =  
fields.Date(index=True) # Filtros por rango de fechas # Campos que NO  
necesitan index: description = fields.Text() # Texto largo, no se busca  
exactamente notes = fields.Html() # HTML, no se indexa internal_comment =  
fields.Char() # Solo para lectura humana
```

Balance

No indexes todo. Los índices aceleran SELECT pero ralentizan INSERT/UPDATE. Solo indexa campos que realmente se usan en búsquedas frecuentes.

4. Seguridad del Código

4.1 SQL Injection

NUNCA concatenes variables directamente en SQL. Siempre usa parámetros.

```
# ❌ PELIGROSO - SQL Injection posible nombre = request.params.get('nombre')
query = "SELECT * FROM res_partner WHERE name = '" + nombre + "'"
self.env.cr.execute(query) # Si nombre = ''; DROP TABLE res_partner; --" 💀
# ✓ SEGURO - Parámetros nombre = request.params.get('nombre') query = "SELECT
* FROM res_partner WHERE name = %s" self.env.cr.execute(query, (nombre,)) #
Tupla de parámetros # ✓ MEJOR - Usa el ORM cuando sea posible partners =
self.env['res.partner'].search([('name', '=', nombre)])
```

4.2 Uso de sudo()

sudo() bypasea TODAS las reglas de seguridad. Úsalo solo cuando sea absolutamente necesario.

```
# ❌ PELIGROSO - sudo innecesario def get_invoices(self): return
self.env['account.move'].sudo().search([]) # Ve TODO # ✓ CORRECTO - Sin
sudo, respeta permisos def get_invoices(self): return
self.env['account.move'].search([]) # Solo lo que debe ver # ✓ JUSTIFICADO
- Usuario portal creando ticket def create_ticket(self): # Usuario portal
no tiene permisos de escritura en helpdesk.ticket # pero DEBE poder crear
su propio ticket return self.env['helpdesk.ticket'].sudo().create({
'name': self.description, 'partner_id': self.env.user.partner_id.id,
})
```

⚠ Regla de Revisión

Cada vez que veas .sudo() en el código, pregunta: '¿Por qué es necesario?' Si la respuesta no es clara y documentada, rechaza el código.

4.3 Validación de Entrada

Nunca confíes en datos que vienen del usuario o de APIs externas.

```
# ✓ CORRECTO - Validación de constraints @api.constrains('discount') def
_check_discount(self): for record in self: if record.discount < 0
or record.discount > 100: raise ValidationError(_("El descuento
debe estar entre 0 y 100")) @api.constrains('date_start', 'date_end') def
_check_dates(self): for record in self: if record.date_end <
record.date_start: raise ValidationError(_("La fecha fin no puede
ser menor a la fecha inicio")) # ✓ CORRECTO - Validación de unicidad en BD
_sql_constraints = [ ('unique_reference', 'UNIQUE(reference)', 'La
referencia debe ser única'), ('positive_quantity', 'CHECK(quantity > 0)',
'La cantidad debe ser positiva'), ]
```

5. Seguridad en Archivos de Configuración

5.1 Permisos de Acceso (ir.model.access.csv)

Este archivo define QUIÉN puede hacer QUÉ en cada modelo. Es OBLIGATORIO.

```
id,name,model_id:id,group_id:id,perm_read,perm_write,perm_create,perm_unlink
access_contrato_user,contrato.user,model_contrato,sales_team.group_sale_salesman,1,1,1,1,0
access_contrato_manager,contrato.manager,model_contrato,sales_team.group_sale_manager,1,1,1,1,1 # Explicación: # - Vendedores (group_sale_salesman): Leer, Escribir, Crear (no Eliminar) # - Gerentes (group_sale_manager): TODO (1,1,1,1)
```

Validación Obligatoria

Si el módulo crea nuevos modelos y NO tiene ir.model.access.csv, RECHAZA el código. Sin este archivo, nadie (excepto admin) podrá acceder al modelo.

5.2 Record Rules (Reglas de Registro)

Las record rules filtran QUÉ registros puede ver cada usuario.

```
<record id="contrato_rule_own" model="ir.rule">    <field name="name">Ver solo mis contratos</field>    <field name="model_id" ref="model_contrato"/>    <field name="domain_force">        [ ('user_id', '=', user.id)]    </field>    <field name="groups" eval="[(4, ref('sales_team.group_sale_salesman'))]" />    </record>    <record id="contrato_rule_all" model="ir.rule">    <field name="name">Ver todos los contratos</field>    <field name="model_id" ref="model_contrato"/>    <field name="domain_force">[(1, '=', 1)]</field>    <!-- Sin filtro -->    <field name="groups" eval="[(4, ref('sales_team.group_sale_manager'))]" />    </record>
```

6. Multi-Compañía

Si el módulo maneja datos transaccionales (ventas, compras, inventario, contabilidad), DEBE respetar la multi-compañía.

6.1 Campo company_id

```
# ✓ CORRECTO - Modelo con soporte multi-compañía class
MiModelo(models.Model):    _name = 'mi.modelo'    _description = 'Mi Modelo'
    company_id = fields.Many2one('res.company', string='Compañía', required=True, default=lambda self: self.env.company, index=True, )    # Record rule automática
_check_company_auto = True    # Valida company_id en relaciones
```

6.2 Validación en Búsquedas

```
# ✗ MAL - Mezcla datos de todas las compañías facturas =
self.env['account.move'].search([]) total =
sum(facturas.mapped('amount_total'))    # ¡Suma Empresa A + B!    # ✓ CORRECTO -
Filtra por compañía actual facturas =
self.env['account.move'].search([ ('company_id', '=', self.env.company.id) ]) total = sum(facturas.mapped('amount_total'))    # ✓
```

MEJOR - Odoo filtra automáticamente si hay record rules # Pero es mejor ser explícito

7. Mantenibilidad y Buenas Prácticas

7.1 Documentación y Comentarios

```
# ✓ CORRECTO - Docstrings completos def calculate_commission(self, amount,
rate):    """    Calcula la comisión basada en el monto y tasa.
Args:        amount (float): Monto base para calcular comisión        rate
(float): Tasa de comisión (0-100)        Returns:        float: Monto de
comisión calculado        Raises:        ValidationError: Si la tasa está
fuera del rango permitido    """    if not 0 <= rate <= 100:        raise
ValidationError(_("La tasa debe estar entre 0 y 100"))        return amount
* (rate / 100)
```

7.2 Traducciones

```
# ✗ MAL - Texto en español fijo raise UserError("El cliente no tiene crédito
disponible") # ✓ CORRECTO - Texto traducible from odoo import _ raise
UserError(_("El cliente no tiene crédito disponible")) # Los mensajes con
_() se pueden traducir a otros idiomas
```

7.3 Manejo de Errores

```
# ✓ CORRECTO - Manejo robusto de errores import logging _logger =
logging.getLogger(__name__) def process_external_api(self):    try:
response = requests.post(url, data=data, timeout=10)
response.raise_for_status()        return response.json()    except
requests.Timeout:        _logger.error("Timeout al conectar con API
externa")        raise UserError(_("La API externa no responde. Intente más
tarde. "))    except requests.HTTPError as e:        _logger.error(f"Error
HTTP: {e}")        raise UserError(_("Error al conectar con la API: %s") %
str(e))    except Exception as e:        _logger.exception("Error
inesperado en API")        raise UserError(_("Error al procesar la
solicitud"))
```

8. Checklist Final para Desarrolladores

Aspecto Técnico	Validado
1. Manifest completo con todas las dependencias	☐
2. Código sigue PEP 8 y convenciones Odoo	☐
3. Métodos CRUD usan super() correctamente	☐
4. No hay queries en loops (problema N+1)	☐
5. Campos computed tienen @api.depends correcto	☐
6. Campos de búsqueda frecuente tienen index=True	☐
7. No hay SQL injection (usa parámetros, no concatenación)	☐
8. sudo() solo usado cuando es necesario y documentado	☐
9. ir.model.access.csv existe y es completo	☐
10. Record rules configuradas correctamente	☐
11. Soporte multi-compañía (company_id donde aplique)	☐
12. Mensajes de error son traducibles (_())	☐
13. Sin IDs hardcodeados (usa XML IDs)	☐
14. Validación de entrada en constraints	☐
15. Logging apropiado en operaciones críticas	☐

PARTE 3: PROCESOS Y HERRAMIENTAS

Tener guías de testing es importante, pero sin un proceso estructurado y herramientas adecuadas, la calidad no será consistente. Esta sección cubre cómo implementar QA en el día a día.

1. Ambientes de Prueba

1.1 Configuración de Ambientes

Sunname Technologies debe mantener al menos 3 ambientes separados:

Ambiente	Propósito	Características
Desarrollo	Desarrolladores externos	Datos dummy, se puede romper
Staging/QA	Testing por Sunname	Copia de producción, testing exhaustivo
Producción	Usuarios finales	Datos reales, NUNCA tocar directamente



Mejores Prácticas

El ambiente de Staging debe ser una COPIA EXACTA de producción (misma versión de Odoo, mismos módulos base, datos anonimizados pero realistas). Si algo funciona en Staging, funcionará en Producción.

2. Flujo de Trabajo Recomendado

2.1 Proceso de Entrega de Módulo

24. Desarrollador externo termina el módulo
25. Desarrollador instala en ambiente de Desarrollo y hace demo básico
26. Sunname recibe el código (Git/ZIP)
27. Desarrollador de Sunname hace revisión de código (Parte 2 de este manual)
28. Si pasa revisión → Instala en Staging
29. Consultor hace testing funcional completo (Parte 1 de este manual)
30. Si pasa testing → Aprobación para producción
31. Se programa ventana de mantenimiento
32. Deploy a producción
33. Monitoreo post-deploy (24-48 horas)



Regla de Oro

NUNCA saltar el paso de Staging. Incluso para 'hot fixes' urgentes, instala primero en

Staging, verifica que funcione, y luego pasa a producción. Un fix mal probado puede causar más daño que el bug original.

3. Herramientas Útiles

3.1 Para Consultores

- Debug mode de Odoo (activar en Configuración)
- Navegador con herramientas de desarrollo (F12)
- Extensión de navegador para validar XML del SAT
- Hoja de cálculo para tracking de casos de prueba
- Herramienta de captura de pantalla (para documentar bugs)

3.2 Para Desarrolladores

- pylint-odoo / flake8 (linters de Python)
- Git para control de versiones
- pgAdmin o DBeaver (explorar base de datos)
- Odoo Shell (acceso CLI al ORM)
- Performance profiler integrado de Odoo

Activar profiler de Odoo (en modo debug): # 1. Activa modo debug con assets
2. Abre cualquier vista # 3. Verás icono de profiler en toolbar # 4.
Activa, realiza acción, revisa SQL queries ejecutados

CONCLUSIÓN

Este manual representa un cambio cultural en Sunname Technologies: pasar de 'corregir bugs en producción' a 'prevenir bugs antes de producción'. No será fácil al principio - requerirá disciplina, tiempo y puede que genere fricciones con desarrolladores externos que no están acostumbrados a este nivel de escrutinio.

Pero los beneficios son claros:

- Menos incidentes en producción = menos estrés para el equipo
- Clientes más satisfechos = mejor reputación
- Código de mayor calidad = menos deuda técnica
- Procesos documentados = más fácil entrenar nuevo personal
- Cultura de calidad = atracción de mejores desarrolladores

La inversión en Quality Assurance se paga sola. Un bug detectado en desarrollo cuesta 1 hora de trabajo. El mismo bug en producción puede costar días de trabajo, pérdida de confianza del cliente, e incluso la pérdida del proyecto completo.

Mensaje Final

Este manual es un documento vivo. A medida que el equipo de Sunname Technologies gane experiencia, encontrará nuevos patrones de errores, mejores técnicas de testing, y casos específicos de su industria. Actualicen este manual regularmente con sus aprendizajes. El conocimiento acumulado es su activo más valioso.

¡Mucho éxito en sus procesos de Quality Assurance!